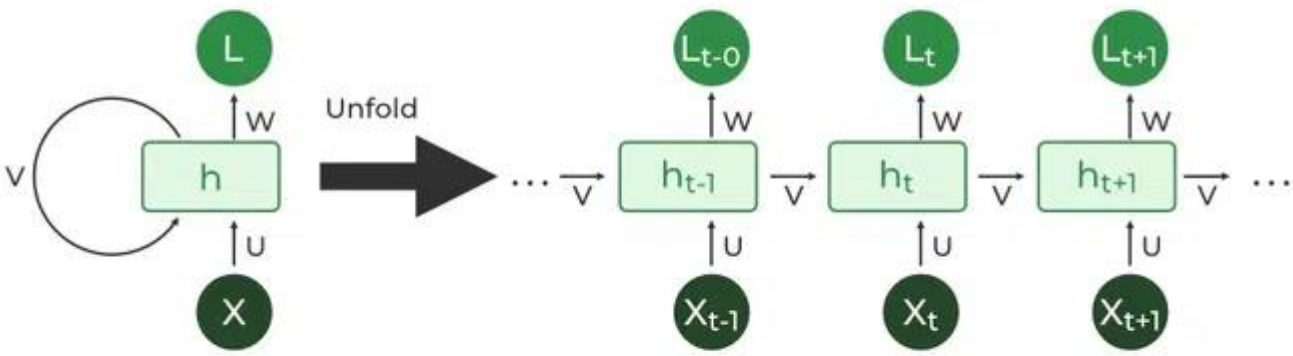
UNIT III

RECURRENT NEURAL NETWORK (RNN) (9 Hrs)

Recurrent Neural Network – Long Short-Term Memory Network – Implementation of RNN & LSTM – Hyperparameters – Embeddings & Word2vec – Sentiment Prediction RNN

RECURRENT NEURAL NETWORK

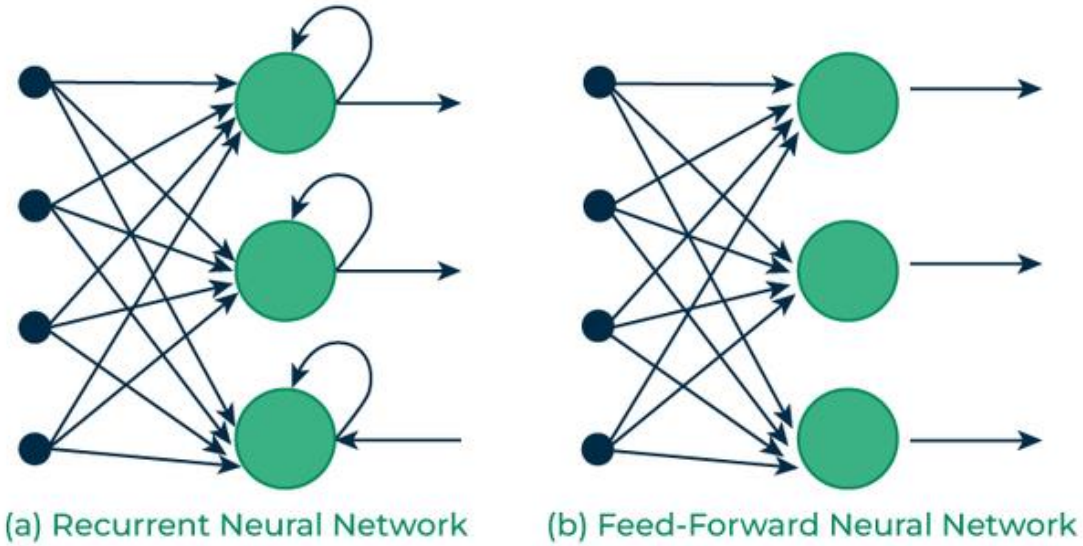
Recurrent Neural Network(RNN) is a type of Neural Network where the output from the previous step is fed as input to the current step. In traditional neural networks, all the inputs and outputs are independent of each other. Still, in cases when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words. Thus RNN came into existence, which solved this issue with the help of a Hidden Layer. The main and most important feature of RNN is its Hidden state, which remembers some information about a sequence. The state is also referred to as *Memory State* since it remembers the previous input to the network. It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output. This reduces the complexity of parameters, unlike other neural networks.



How RNN differs from Feedforward Neural Network?

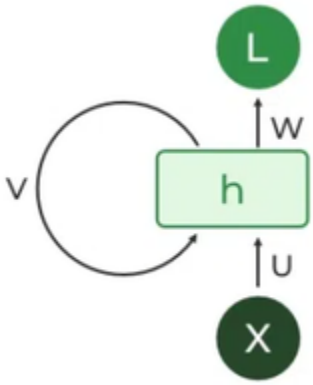
Artificial neural networks that do not have looping nodes are called feed forward neural networks. Because all information is only passed forward, this kind of neural network is also referred to as a multi-layer neural network.

Information moves from the input layer to the output layer – if any hidden layers are present – unidirectionally in a feedforward neural network. These networks are appropriate for image classification tasks, for example, where input and output are independent. Nevertheless, their inability to retain previous inputs automatically renders them less useful for sequential data analysis.



Recurrent Neuron and RNN Unfolding

The fundamental processing unit in a Recurrent Neural Network (RNN) is a Recurrent Unit, which is not explicitly called a "Recurrent Neuron." This unit has the unique ability to maintain a hidden state, allowing the network to capture sequential dependencies by remembering previous inputs while processing. Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) versions improve the RNN's ability to handle long-term dependencies.



L – long short term

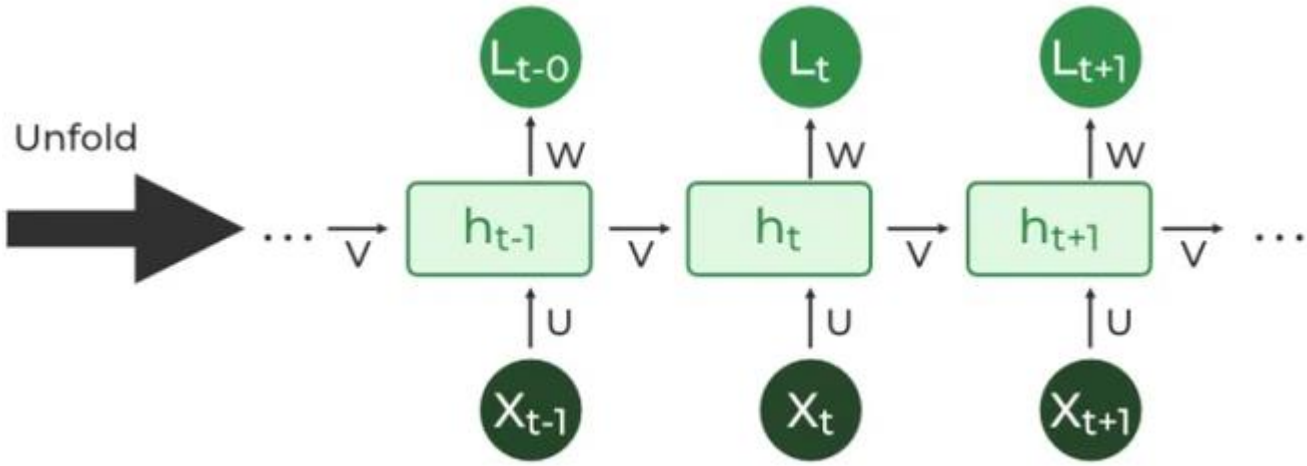
W – weight

h – current state

U – hidden state

V – output state

X – input state



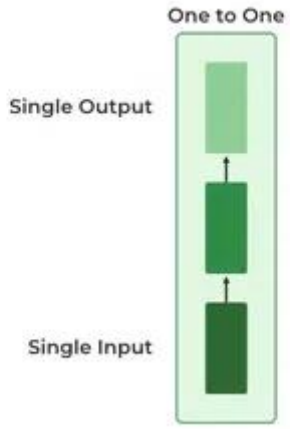
Types Of RNN

There are four types of RNNs based on the number of inputs and outputs in the network.

1. One to One
2. One to Many
3. Many to One
4. Many to Many

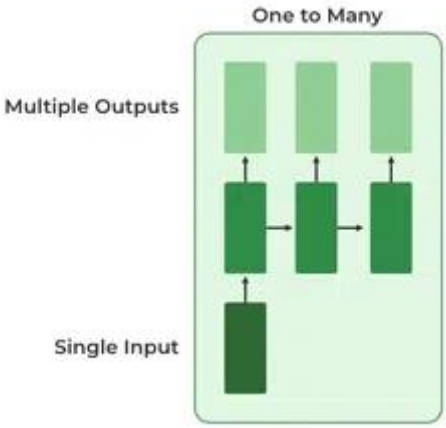
One to One

This type of RNN behaves the same as any simple Neural network it is also known as Vanilla Neural Network. In this Neural network, there is only one input and one output.



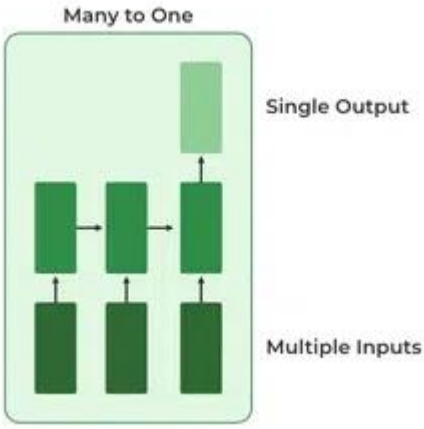
One To Many

In this type of RNN, there is one input and many outputs associated with it. One of the most used examples of this network is Image captioning where given an image we predict a sentence having Multiple words.



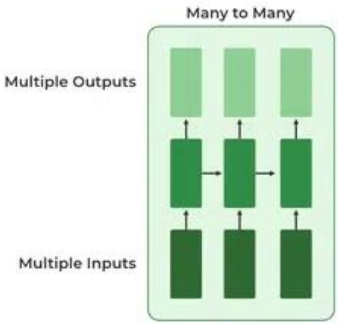
Many to One

In this type of network, Many inputs are fed to the network at several states of the network generating only one output. This type of network is used in the problems like sentimental analysis. Where we give multiple words as input and predict only the sentiment of the sentence as output.



Many to Many

In this type of neural network, there are multiple inputs and multiple outputs corresponding to a problem. One Example of this Problem will be language translation. In language translation, we provide multiple words from one language as input and predict multiple words from the second language as output.



Recurrent Neural Network Architecture

RNNs have the same input and output architecture as any other deep neural architecture. However, differences arise in the way information flows from input to output. Unlike Deep neural networks where we have different weight matrices for each Dense network in RNN, the

weight across the network remains the same. It calculates state hidden state **Hi** for every input **Xi . By using the following formulas:**

h= σ(UX + Wh-1 + B)

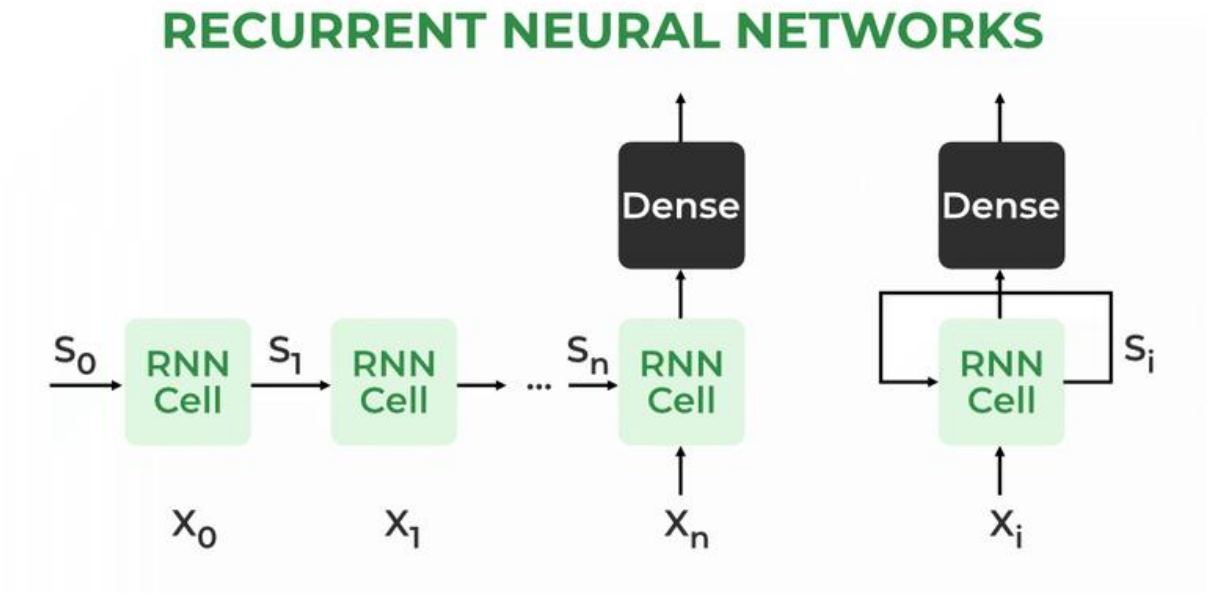
Y = O(Vh + C)

Hence

Y = f (X, h , W, U, V, B, C)

Here S is the State matrix which has element si as the state of the network at timestep i

The parameters in the network are W, U, V, c, b which are shared across timestep



How does RNN work?

The Recurrent Neural Network consists of multiple fixed activation function units, one for each time step. Each unit has an internal state which is called the hidden state of the unit. This hidden state signifies the past knowledge that the network currently holds at a given time step. This hidden state is updated at every time step to signify the change in the knowledge of the network about the past. The hidden state is updated using the following recurrence relation:-

The formula for calculating the current state:

where,

- ht -> current state
- ht-1 -> previous state
- xt -> input state

Formula for applying Activation function(tanh)

where,

- whh -> weight at recurrent neuron
- wxh -> weight at input neuron

The formula for calculating output:

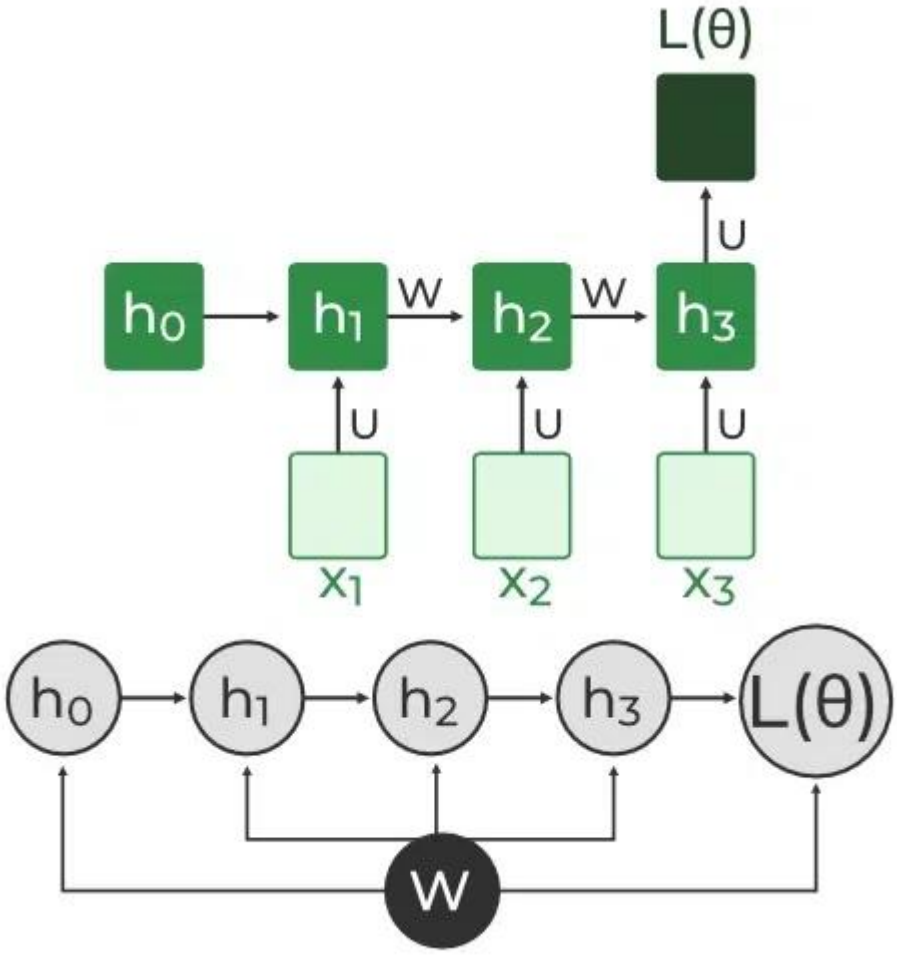
- Yt -> output
- Why -> weight at output layer

These parameters are updated using Backpropagation. However, since RNN works on sequential data here we use an updated backpropagation which is known as Backpropagation through time.

Backpropagation Through Time (BPTT)

In RNN the neural network is in an ordered fashion and since in the ordered network each variable is computed one at a time in a specified order like first h1 then h2 then h3 so on. Hence we will apply backpropagation throughout all these hidden time states sequentially.

- L(θ)(loss function) depends on h3
- h3 in turn depends on h2 and W
- h2 in turn depends on h1 and W
- h1 in turn depends on h0 and W
- where h0 is a constant starting state.



For simplicity of this equation, we will apply backpropagation on only one row

We already know how to compute this one as it is the same as any simple deep neural network backpropagation.

.However, we will see how to apply backpropagation to this term

As we know $h3 = \sigma(Wh2 + b)$

And In such an ordered network, we can't compute by simply treating h3 as a constant because as it also depends on W. the total derivative has two parts:

1. **Explicit:** treating all other inputs as constant
2. **Implicit:** Summing over all indirect paths from h3 to W

This algorithm is called backpropagation through time (BPTT) as we backpropagate over all previous time steps

Issues of Standard RNNs

1. **Vanishing Gradient:** Text generation, machine translation, and stock market prediction are just a few examples of the time-dependent and sequential data problems that can be modelled with recurrent neural networks. You will discover, though, that the gradient problem makes training RNN difficult.
2. **Exploding Gradient:** An Exploding Gradient occurs when a neural network is being trained and the slope tends to grow exponentially rather than decay. Large error gradients that build up during training lead to very large updates to the neural network model weights, which is the source of this issue.

Training through RNN

1. A single-time step of the input is provided to the network.
2. Then calculate its current state using a set of current input and the previous state.
3. The current ht becomes ht-1 for the next time step.
4. One can go as many time steps according to the problem and join the information from all the previous states.
5. Once all the time steps are completed the final current state is used to calculate the output.
6. The output is then compared to the actual output i.e the target output and the error is generated.
7. The error is then back-propagated to the network to update the weights and hence the network (RNN) is trained using Backpropagation through time.

Advantages and Disadvantages of Recurrent Neural Network

Advantages

1. An RNN remembers each and every piece of information through time. It is useful in time series prediction only because of the feature to remember previous inputs as well. This is called Long Short Term Memory.
2. Recurrent neural networks are even used with convolutional layers to extend the effective pixel neighborhood.

Disadvantages

1. Gradient vanishing and exploding problems.
2. Training an RNN is a very difficult task.
3. It cannot process very long sequences if using tanh or relu as an activation function.

Applications of Recurrent Neural Network

1. Language Modelling and Generating Text
2. Speech Recognition
3. Machine Translation
4. Image Recognition, Face detection
5. Time series Forecasting

Variation Of Recurrent Neural Network (RNN)

To overcome the problems like vanishing gradient and exploding gradient descent several new advanced versions of RNNs are formed some of these are as;

1. Bidirectional Neural Network (BiNN)
2. Long Short-Term Memory (LSTM)

Bidirectional Neural Network (BiNN)

A BiNN is a variation of a Recurrent Neural Network in which the input information flows in both direction and then the output of both direction are combined to produce the input. BiNN is useful in situations when the context of the input is more important such as Nlp tasks and Time-series analysis problems.

Long Short-Term Memory (LSTM)

Long Short-Term Memory works on the read-write-and-forget principle where given the input information network reads and writes the most useful information from the data and it forgets about the information which is not important in predicting the output. For doing this three new gates are introduced in the RNN. In this way, only the selected information is passed through the network.

Long Short Term Memory

Long Short-Term Memory is an improved version of recurrent neural network designed by Hochreiter & Schmidhuber. **LSTM** is well-suited for sequence prediction tasks and excels in capturing long-term dependencies. Its applications extend to tasks involving time series and sequences. LSTM's strength lies in its ability to grasp the order dependence crucial for solving intricate problems, such as machine translation and speech recognition. The article provides an in-depth introduction to LSTM, covering the LSTM model, architecture, working principles, and the critical role they play in various applications.

What is LSTM?

A traditional RNN has a single hidden state that is passed through time, which can make it difficult for the network to learn long-term dependencies. LSTMs address this problem by introducing a memory cell, which is a container that can hold information for an extended period. LSTM networks are capable of learning long-term dependencies in sequential data, which makes them well-suited for tasks such as language translation, speech recognition, and time series forecasting. LSTMs can also be used in combination with other neural network architectures, such as Convolutional Neural Networks (CNNs) for image and video analysis.

The memory cell is controlled by three gates: the input gate, the forget gate, and the output gate. These gates decide what information to add to, remove from, and output from the memory cell. The input gate controls what information is added to the memory cell. The forget gate controls what information is removed from the memory cell. And the output gate controls what information is output from the memory cell. This allows LSTM networks to selectively retain or discard information as it flows through the network, which allows them to learn long-term dependencies.

Bidirectional LSTM

Bidirectional LSTM (Bi LSTM/ BLSTM) is recurrent neural network (RNN) that is able to process sequential data in both forward and backward directions. This allows Bi LSTM to learn longer-range dependencies in sequential data than traditional LSTMs, which can only process sequential data in one direction.

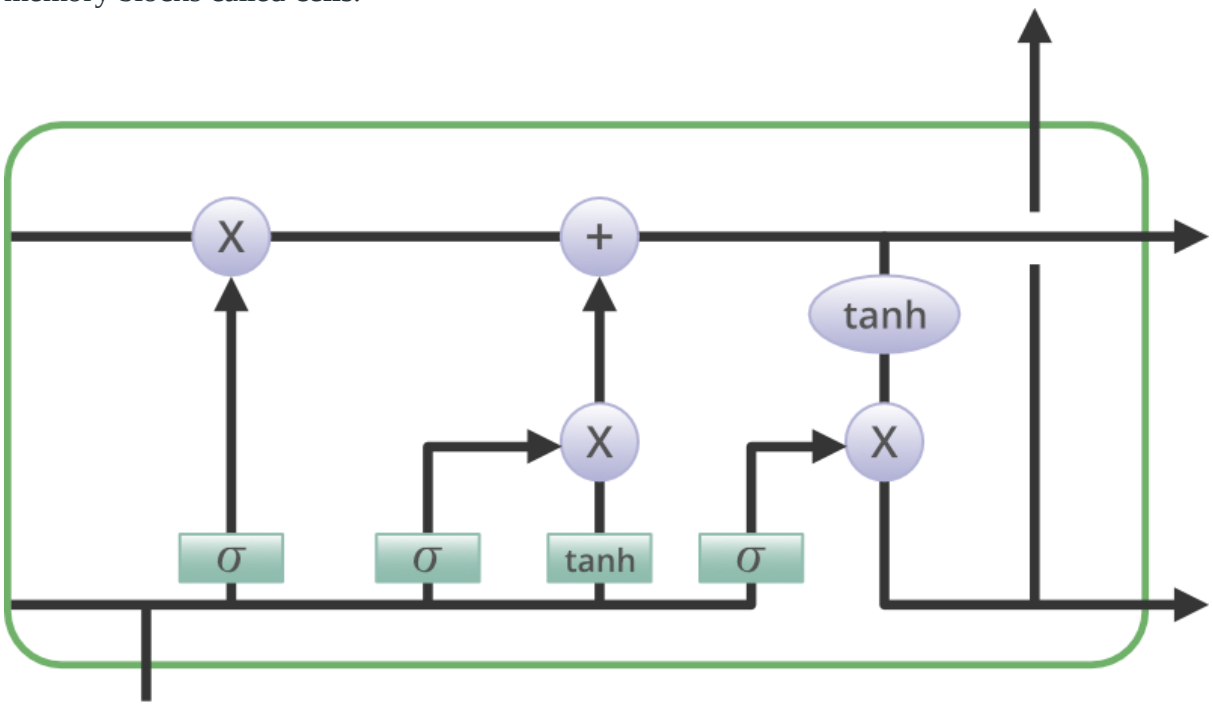
- Bi LSTMs are made up of two LSTM networks, one that processes the input sequence in the forward direction and one that processes the input sequence in the backward direction. The outputs of the two LSTM networks are then combined to produce the final output.

- Bi LSTM have been shown to achieve state-of-the-art results on a wide variety of tasks, including machine translation, speech recognition, and text summarization.

LSTMs can be stacked to create deep LSTM networks, which can learn even more complex patterns in sequential data. Each LSTM layer captures different levels of abstraction and temporal dependencies in the input data.

Architecture and Working of LSTM

LSTM architecture has a chain structure that contains four neural networks and different memory blocks called **cells**.



nformation is retained by the cells and the memory manipulations are done by the **gates.** There are three gates –

Forget Gate

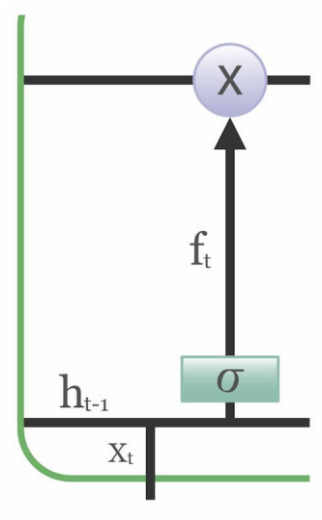
The information that is no longer useful in the cell state is removed with the forget gate. Two inputs *xt* (input at the particular time) and *ht-1* (previous cell output) are fed to the gate and multiplied with weight matrices followed by the addition of bias. The resultant is passed through an activation function which gives a binary output. If for a particular cell state the output is 0, the piece of information is forgotten and for output 1, the information is retained for future use.

The equation for the forget gate is:

$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$

where:

- W_f represents the weight matrix associated with the forget gate.
- $[h_{t-1}, x_t]$ denotes the concatenation of the current input and the previous hidden state.
- b_f is the bias with the forget gate.
- σ is the sigmoid activation function.



Input gate

The addition of useful information to the cell state is done by the input gate. First, the information is regulated using the sigmoid function and filter the values to be remembered similar to the forget gate using inputs *ht-1* and *xt*. . Then, a vector is created using *tanh* function that gives an output from -1 to +1, which contains all the possible values from ht-1 and *xt*. At last, the values of the vector and the regulated values are multiplied to obtain the useful information. The equation for the input gate is:

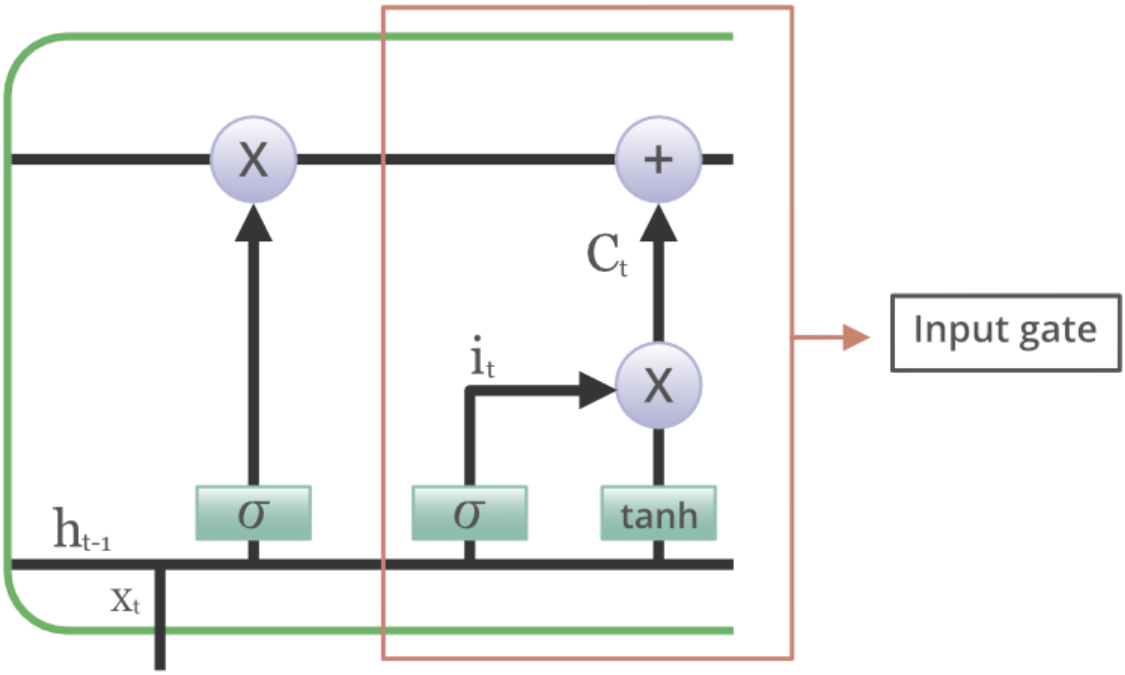
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\hat{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \hat{C}_t$$

where

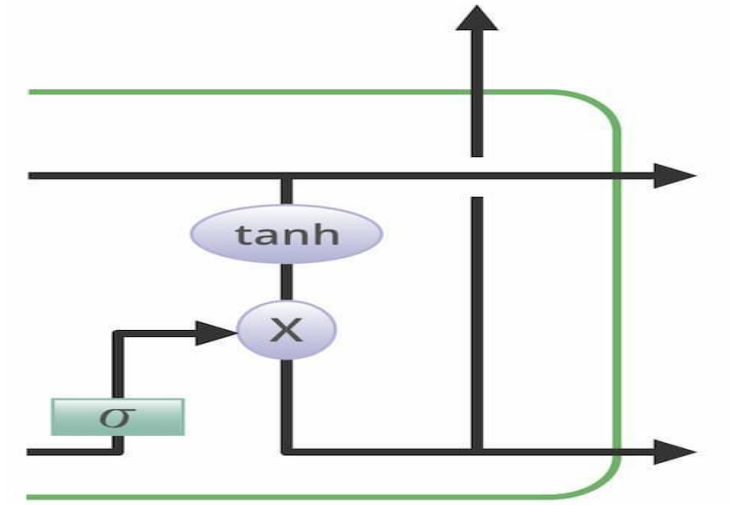
- $\odot$ denotes element-wise multiplication
- tanh is tanh activation function



Output gate

The task of extracting useful information from the current cell state to be presented as output is done by the output gate. First, a vector is generated by applying tanh function on the cell. Then, the information is regulated using the sigmoid function and filter by the values to be remembered using inputs *ht-1* and *xt*. At last, the values of the vector and the regulated values are multiplied to be sent as an output and input to the next cell. The equation for the output gate is:

o_t = σ(W_o · [h_{t-1}, x_t] + b_o)



The advantages of LSTM (Long-Short Term Memory) are as follows:

- Long-term dependencies can be captured by LSTM networks. They have a memory cell that is capable of long-term information storage.
- In traditional RNNs, there is a problem of vanishing and exploding gradients when models are trained over long sequences. By using a gating mechanism that selectively recalls or forgets information, LSTM networks deal with this problem.

The disadvantages of LSTM (Long-Short Term Memory) are as follows:

- Compared to simpler architectures like feed-forward neural networks LSTM networks are computationally more expensive. This can limit their scalability for large-scale datasets or constrained environments.
- Training LSTM networks can be more time-consuming compared to simpler models due to their computational complexity. So training LSTMs often requires more data and longer training times to achieve high performance.

Applications of LSTM

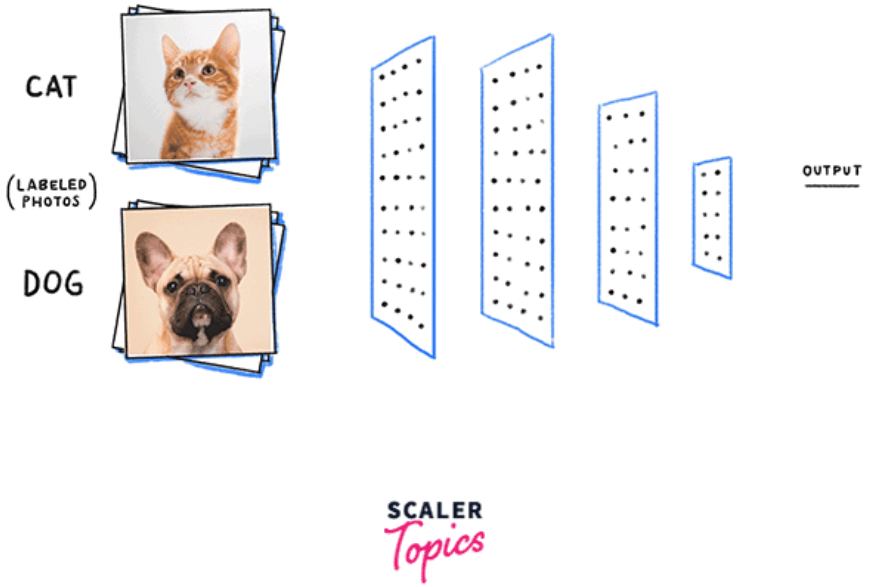
Some of the famous applications of LSTM includes:

- **Language Modeling:** LSTMs have been used for natural language processing tasks such as language modeling, machine translation, and text summarization. They can be trained to generate coherent and grammatically correct sentences by learning the dependencies between words in a sentence.
- **Speech Recognition:** LSTMs have been used for speech recognition tasks such as transcribing speech to text and recognizing spoken commands. They can be trained to recognize patterns in speech and match them to the corresponding text.
- **Time Series Forecasting:** LSTMs have been used for time series forecasting tasks such as predicting stock prices, weather, and energy consumption. They can learn patterns in time series data and use them to make predictions about future events.

Hyperparameters in Deep Learning

Introduction

Deep learning, a subset of machine learning, has revolutionized various fields, from natural language processing to image recognition. At the heart of deep learning's success is the nuanced and often complex process of training deep neural networks. This training is heavily influenced by hyperparameters, adjustable settings that dictate the model's learning process. Unlike parameters, which the model learns through training, hyperparameters are preset by the practitioner and remain constant during the training phase. Understanding and optimizing these hyperparameters is crucial for building effective deep learning models.



The Importance of Hyperparameters

Hyperparameters play a pivotal role in the performance of deep learning models. They impact how quickly a model learns, its ability to generalize from training data to unseen data, and its overall performance. The right set of hyperparameters can mean the difference between a mediocre model and a highly accurate one. However, determining the optimal hyperparameters is not straightforward. It requires a deep understanding of both the theoretical aspects of deep learning and practical experience.

Key Hyperparameters in Deep Learning

Hyperparameters in deep learning are adjustable parameters that you choose to train a model that govern the training process itself. Unlike model parameters, which are learned during training, hyperparameters are set prior to the training process and remain constant during training. They can significantly influence the performance of your trained models. Here are some of the most common hyperparameters in deep learning:

1. **Learning Rate:** Perhaps the most critical hyperparameter, the learning rate determines the step size at each iteration while moving toward a minimum of the loss function.

2. **Number of Epochs:** This is the number of times the learning algorithm will work through the entire training dataset.

3. **Batch Size:** It refers to the number of training samples used in one iteration.

4. **Number of Layers and Neurons in Each Layer:** In neural networks, deciding on the number of layers and the number of neurons in each layer can significantly impact the model's performance.

5. **Activation Function:** Functions like ReLU, Sigmoid, and Tanh, determine the output of a neural network node given an input or set of inputs.

6. **Optimizer**: Algorithms like SGD, Adam, and RMSprop are used to change the attributes of the neural network such as weights and learning rate to reduce the losses.

7. **Dropout Rate:** Used in dropout layers to prevent overfitting, it determines the fraction of the output units to drop at each training stage.

8. **Weight Initialization:** This defines the method to set the initial random weights of Keras layers.

9. **Momentum**: Often used in optimization algorithms, it helps to accelerate gradient vectors in the right directions, thus leading to faster converging.

10. **Regularization Techniques**: Techniques like L1 and L2 regularization to prevent overfitting by penalizing complex models.

The choice of hyperparameters can vary widely depending on the specific problem and data set. Selecting the right set of hyperparameters is more of an art than a science, often involving a fair bit of trial and error or more systematic approaches like grid search or random search.

Challenges in Hyperparameter Tuning

Choosing the right hyperparameters is often challenging. It's a multidimensional problem where each hyperparameter can interact with others in unpredictable ways. Manual tuning is time-consuming and requires extensive experience. Therefore, automated methods like grid search, random search, and Bayesian optimization are increasingly popular. These methods systematically explore the hyperparameter space to find the most effective combination.

Implementation of RNN and LSTM:

Introduction

In NLP we have seen some NLP tasks using traditional neural networks, like text classification, sentiment analysis, and we did it with satisfactory results. but this wasn't enough, we faced certain problems with traditional neural networks as given below.

- The prediction was highly dependent on the specific words.

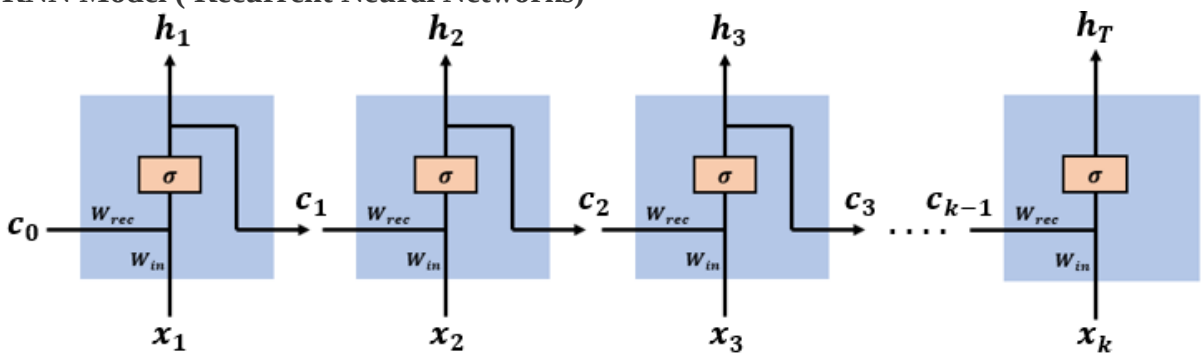
- If you change any word with it synonyms result gets deviated.3. If you write *like*, *affection* instead of *love*, the result gets deviated.4. The prediction was not dependent on the sequence of words.

To fix this problem we came up with the idea of Word Embedding and a model which can store the sequence of the words and depending on the sequence it can generate results.

*ie "**I love playing Ball**" and "**I do not love playing Ball**"*

In both the sentence we see "love" hence our vanilla model can classify it as a positive sentiment but as you see its negative sentiment, here sequence of words matters a lot. So far we had no model which can consider the sequence of words in a sentence for the prediction.

RNN Model (Recurrent Neural Networks)



To solve this problem **Recurrent neural network** came into the picture. which solves this problem by using hidden layers. and hidden layers are the main features of a recurrent neural network. hidden layers help RNN to remember the sequence of words (data) and use the sequence pattern for the prediction.

The idea of RNN has developed a lot. it has been used for speech recognition and various NLP tasks where the sequence of words matters. RNN takes input as time series (sequence of words), we can say RNN acts like a memory that remembers the sequence.

Use cases of RNN

Speech, text recognition & sentiment classificationMusic synthesisImage captioningChatbots & NLP(natural language processing)Machine Translation—Language translationStock predictionsFinding and blocking abusive words in a speech or text

Does RNN sound so powerful? No ! there is a big problem.

Problems with RNN :

Exploding and vanishing gradient problems during backpropagation.

Gradients are those values which to update neural networks weights. In other words, we can say that Gradient carries information.

Vanishing gradient is a big problem in deep neural networks. it vanishes or explodes quickly in earlier layers and this makes RNN unable to hold information of longer sequence. and thus **RNN becomes short-term memory**.

If we apply RNN for a paragraph RNN may leave out necessary information due to gradient problems and not be able to carry information from the initial time step to later time steps.

To solve this problem LSTM, GRU came into the picture.

How do LSTM, GRU solve this problem?

I highly encourage you to read Colah's blog for in-depth knowledge of LSTM.

The reason for exploding gradient was the capturing of relevant and irrelevant information. a model which can decide what information from a paragraph and relevant and remember only relevant information and throw all the irrelevant information

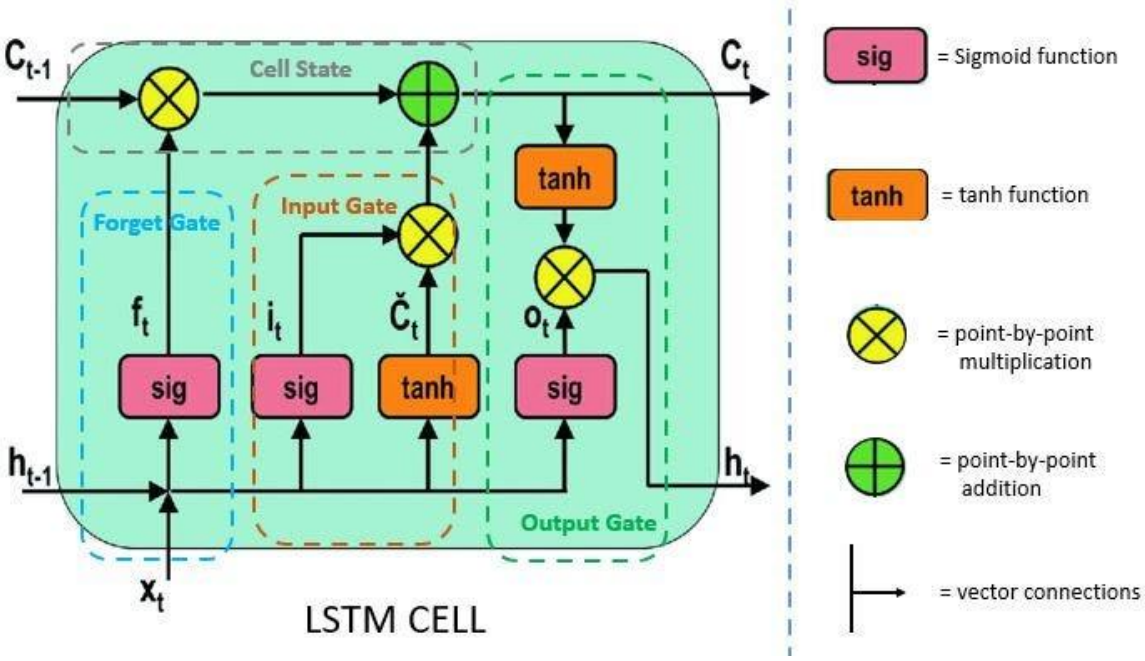
This is achieved by using gates. the LSTM (Long -short-term memory) and GRU (Gated Recurrent Unit) have gates as an internal mechanism, which control what information to keep and what information to throw out. By doing this LSTM, GRU networks solve the exploding and vanishing gradient problem.

Almost each and every SOTA (state of the art) model based on **RNN** follows LSTM or GRU networks for prediction.

LSTMs /GRUs are implemented in speech recognition, text generation, caption generation, etc.

LSTM networks Step by Step

Every LSTM network basically contains three gates to control the flow of information and cells to hold information. The Cell States carries the information from initial to later time steps without getting vanished.



Gates

Gates make use of sigmoid activation or you can say *tanh* activation. values ranges in *tanh* activation are 0 -1.

1. Forget Gate
2. Input Gate
3. Output Gate

Let's see these gates in detail:

Forget Gate

This gate decides what information should be carried out forward or what information should be ignored.

Information from previous hidden states and the current state information passes through the sigmoid function. Values that come out from sigmoid are always between 0 and 1. if the value is closer to 1 means information should proceed forward and if value closer to 0 means information should be ignored.

Input Gate

After deciding the relevant information, the information goes to the input gate, Input gate passes the relevant information, and this leads to updating the cell states. simply saving updating the weight.

Input gate adds the new relevant information to the existing information by updating cell states.

Output Gate

After the information is passed through the input gate, now the output gate comes into play. Output gate generates the next hidden states. and cell states are carried over the next time step.

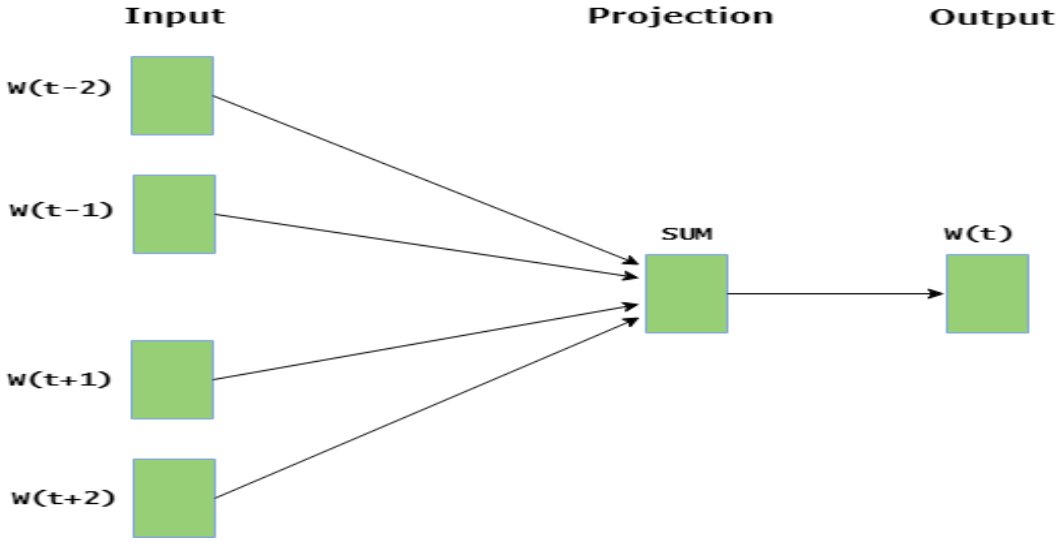
EMBEDDINGS AND WORD2VEC:

What is Word Embedding?
Word Embedding is a language modeling technique for mapping words to vectors of real numbers. It represents words or phrases in vector space with several dimensions. Word embeddings can be generated using various methods like neural networks, co-occurrence matrices, probabilistic models, etc. **Word2Vec** consists of models for generating word embedding. These models are shallow two-layer neural networks having one input layer, one hidden layer, and one output layer.

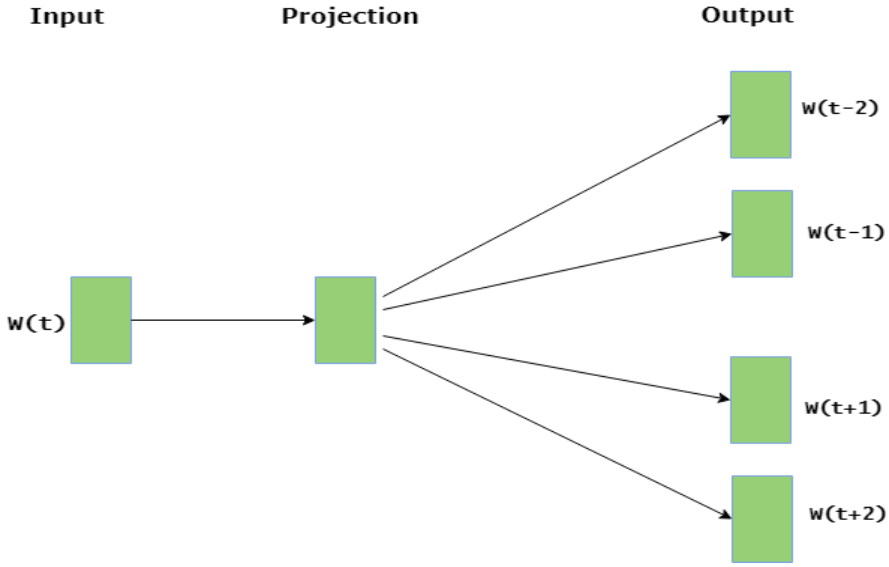
What is Word2Vec?
Word2Vec is a widely used method in natural language processing (NLP) that allows words to be represented as vectors in a continuous vector space. Word2Vec is an effort to map words to high-dimensional vectors to capture the semantic relationships between words, developed by

researchers at Google. Words with similar meanings should have similar vector representations, according to the main principle of Word2Vec. Word2Vec utilizes two architectures:

- **CBOW (Continuous Bag of Words):** The CBOW model predicts the current word given context words within a specific window. The input layer contains the context words and the output layer contains the current word. The hidden layer contains the dimensions we want to represent the current word present at the output layer.



- **Skip Gram :** Skip gram predicts the surrounding context words within specific window given current word. The input layer contains the current word and the output layer contains the context words. The hidden layer contains the number of dimensions in which we want to represent current word present at the input layer.



The basic idea of word embedding is words that occur in similar context tend to be closer to each other in vector space. For generating word vectors in Python, modules needed are **nltk** and **gensim**. Run these commands in terminal to install **nltk** and **gensim**:

```
pip install nltk
pip install gensim
```

- **NLTK:** For handling human language data, NLTK, or Natural Language Toolkit, is a potent Python library. It offers user-friendly interfaces to more than 50 lexical resources and corpora, including WordNet. A collection of text processing libraries for tasks like categorization, tokenization, stemming, tagging, parsing, and semantic reasoning are also included with NLTK.
- **GENSIM:** Gensim is an open-source Python library that uses topic modelling and document similarity modelling to manage and analyse massive amounts of unstructured text data. It is especially well-known for applying topic and vector space modelling algorithms, such as Word2Vec and Latent Dirichlet Allocation (LDA), which are widely used.

Why we need Word2Vec?

In natural language processing (NLP), Word2Vec is a popular and significant method for representing words as vectors in a continuous vector space. Word2Vec has become popular and is utilized in many different NLP applications for several reasons:

- **Semantic Representations:** Word2Vec records the connections between words semantically. Words are represented in the vector space so that similar words are near to one another. This enables the model to interpret words according to their context within a particular corpus.
- **Distributional Semantics:** The foundation of Word2Vec is the distributional hypothesis, which holds that words with similar meanings are more likely to occur in similar contexts. Word2Vec generates vector representations that reflect semantic similarities by learning from the distributional patterns of words in a large corpus.
- **Vector Arithmetic:** Word2Vec generates vector representations that have intriguing algebraic characteristics. Vector arithmetic, for instance, can be used to record word relationships. One well-known example is that the vector representation of “queen” could resemble the vector representation of “king” less “man” plus “woman.”
- **Efficiency:** Word2Vec’s high computational efficiency makes training on big datasets possible. Learning high-dimensional vector representations for a large vocabulary requires this efficiency.
- **Transfer Learning:** A variety of natural language processing tasks can be initiated with pre-trained Word2Vec models. Time and resources can be saved by fine-tuning the embeddings discovered on a sizable dataset for particular uses.
- **Applications:** Word2Vec embeddings have shown promise in a number of natural language processing (NLP) applications, such as machine translation, text classification, sentiment analysis, and information retrieval. These applications are successful in part because of their capacity to capture semantic relationships.
- **Scalability:** Word2Vec can handle big corpora with ease and is scalable. Scalability like this is essential for training on large text datasets.
- **Open Source Implementations:** Word2Vec has open-source versions, including one that is included in the Gensim library. Its widespread adoption and use in both research and industry can be attributed in part to its accessibility.

SENTIMENT PREDICTION RNN:

Building a sentiment prediction Recurrent Neural Network (RNN) involves using a type of neural network architecture that can handle sequential data, such as text. RNNs, with their ability to retain information from previous steps, are particularly well-suited for tasks like

sentiment analysis. Here's a basic outline of how you can approach building a sentiment prediction RNN:

1. **Data Preparation**:
    - Obtain a dataset labeled with sentiment (positive, negative, neutral).
    - Preprocess the text data: tokenization, removing stop words, punctuation, lowercasing, etc.
    - Convert words into numerical representations (word embeddings).
2. **Model Architecture**:
    - Define the architecture of your RNN model. You can use different types of RNN cells like vanilla RNN, LSTM, or GRU. LSTM and GRU are preferred choices due to their ability to handle long-range dependencies.
    - Decide whether to use pre-trained word embeddings (like Word2Vec, GloVe) or train embeddings from scratch.
    - Include an output layer with appropriate activation function (sigmoid for binary classification, softmax for multi-class classification).
    - Consider adding layers for regularization (dropout) to prevent overfitting.
3. **Model Training**:
    - Split your dataset into training, validation, and testing sets.
    - Train your RNN model using the training data. Monitor performance on the validation set to avoid overfitting.
    - Experiment with different hyperparameters such as learning rate, batch size, number of epochs, etc.
4. **Model Evaluation**:
    - Evaluate the trained model on the test set using appropriate metrics (accuracy, precision, recall, F1-score).
    - Analyze the model's performance and fine-tune hyperparameters if necessary.
5. **Deployment**:
    - Once satisfied with the model's performance, deploy it for inference on new data.
    - Monitor the model's performance in the production environment and update as needed.